

T1.3.C — procesar_evento_suscripcion + idempotencia

Implementación · Rama `pr/t1.3.c-procesar-evento-suscripcion` · Commit `5e00393` · 2026-05-02

Status: Branch pusheado a GitHub. Tests verdes (33 nuevos · 528 totales). **NO mergeado a main.** Owner aprueba via PR en GitHub UI.

Dependencia de orden: Esta rama incluye los commits de T1.3.A (modelos) y T1.3.B (helper) además del de C. Se requiere mergear T1.3.A y T1.3.B a main **primero**; cuando eso pase, GitHub recalcula el diff del PR de C y solo mostrará los cambios netos de T1.3.C.

Qué se implementó

Se agregó el dispatcher `procesar_evento_suscripcion(evento)` en `agent/billing.py` que maneja los 4 tipos de evento del ciclo de vida de una suscripción Stripe en Dona, junto con 4 helpers privados (uno por tipo). El dispatcher implementa idempotencia a tres niveles.

Función pública: `procesar_evento_suscripcion`

```
async def procesar_evento_suscripcion(evento: dict) -> dict
    Maneja:
        checkout.session.completed (mode=subscription)
        invoice.payment_succeeded
        customer.subscription.updated
        customer.subscription.deleted
    Idempotencia general por evento.id antes de despachar.
    Retorna dict {handled: bool, ...contexto}.
```

Helpers privados

- `_procesar_checkout_subscription(data)`: crea o actualiza fila `SuscripcionStripe`. **NO acredita** — espera `invoice.payment_succeeded` para evitar regalar créditos si el cobro nunca se concreta.
- `_procesar_invoice_payment_succeeded(data)`: acredita `creditos_mensuales` al telefono de la suscripción. Idempotente por `ultimo_invoice_acreditado` y por `TransaccionCredito.stripe_session_id`.
- `_procesar_subscription_updated(data)`: actualiza status / price_id / plan. Si cambia el plan_codigo, recalcula `creditos_mensuales`. NO toca saldo.
- `_procesar_subscription_deleted(data)`: marca status='canceled'. PRESERVA el saldo del usuario (decisión owner).

Decisiones owner reflejadas en código

Decisión	Implementación
Premium=100, Pro=500	Tomado de <code>creditos_de_plan()</code> (T1.3.B)
Renovaciones acumulables	<code>acreditar()</code> suma al saldo, no resetea
Checkout no acredita	Solo crea/actualiza fila, espera invoice
Cancelar preserva saldo	<code>subscription.deleted</code> no toca <code>SaldoCredito</code> s

Upgrade/downgrade aplica al próximo periodo subscription.updated no toca saldo

Eventos no manejados se reintentan

Solo marcamos handled=True en EventoStripeProcesado

Idempotencia: 3 niveles de defensa

Stripe reentrega webhooks tras cualquier respuesta no-2xx. Tres llaves diferentes nos protegen contra distintos modos de duplicación:

Nivel	Llave	Tabla	Cuándo aplica
1	event.id	EventoStripeProcesado	Cualquier evento ya procesado
2	invoice.id	SuscripcionStripe.ultimo_invoice_acreditado	Solo invoices (event_id distinto, mismo invoice)
3	stripe_session_id (=invoice_id)	TransaccionCredito	Última red de seguridad dentro de acreditar()

Política: solo marcamos eventos exitosos

Si un evento falla por config faltante (plan desconocido, env sin configurar, telefono ausente), **no** escribimos en EventoStripeProcesado. Razón: si después arreglas la config y haces 'replay' en Stripe Dashboard, el evento se vuelve a procesar y acredita correctamente. Si lo marcáramos como procesado, la corrección requeriría intervención manual.

Tests · 33 nuevos · 528 totales

Archivo nuevo: **tests/test_procesar_evento_suscripcion.py** (641 líneas). Patrón fixture idéntico a test_billing.py: SQLite en tmp_path con monkeypatch.setenv + importlib.reload.

Clase de tests	Cantidad	Cubre
TestCheckoutSessionCompleted	9	Crea/actualiza, no acredita aún, mode=payment ignorado, telefono fallback, plan desconocido,
TestInvoicePaymentSucceeded	8	Acredita, invoice duplicado, event_id duplicado, renovaciones acumulables, race con checkout
TestSubscriptionUpdated	5	status/plan/price, recalc creditos_mensuales, no toca saldo, plan desconocido
TestSubscriptionDeleted	3	Marca canceled, preserva saldo, sub_id desconocido
TestIdempotenciaEventId	4	event.id duplicado skip, no-handled no se marca, handled sí se marca, sin id rechaza
TestTipoNoSoportado	2	tipo random no revienta, payment_failed (T1.3.C scope)
TestLegacyOneTimeIntacto	1	procesar_evento_stripe legacy paquetes one-time sigue OK

Resultado

```
$ pytest --tb=short -q
...
528 passed, 5 warnings in 152.05s (0:02:32)

Baseline post-T1.3.B: 488 tests
Δ = +40 tests (33 T1.3.C + 7 T1.3.A reactivados por cherry-pick)
```

Archivos modificados

Archivo	Δ líneas	Tipo
agent/billing.py	+406 / -0	M
tests/test_procesar_evento_suscripcion.py	+641 / -0	A
TOTAL T1.3.C	+1047 / -0	2 archivos

Los archivos de T1.3.A (alembic/versions/002_suscripcion_stripe.py, agent/memory.py +46, tests/test_suscripcion_stripe_models.py) y T1.3.B (.env.example +17, tests/test_creditos_de_plan.py) figuran en commits previos del branch (cherry-picked desde sus PRs originales).

Próximos pasos para el owner

1. Mergear T1.3.A y T1.3.B primero (orden importa)

El PR de C en GitHub mostrará un diff que incluye los 3 commits hasta que A y B estén en main. Una vez mergeados:

- **PR T1.3.A:** <https://github.com/celestinojbm/Dona-agent/compare/main...pr/t1.3.a-suscripcion-stripe-modelos>
- **PR T1.3.B:** <https://github.com/celestinojbm/Dona-agent/compare/main...pr/t1.3.b-creditos-de-plan-helper>

2. Mergear T1.3.C (después de A y B)

Una vez A y B estén en main, el diff del PR de C se reduce automáticamente a los 2 archivos de T1.3.C (+1047 líneas):

- **PR T1.3.C:** <https://github.com/celestinojbm/Dona-agent/compare/main...pr/t1.3.c-procesar-evento-suscripcion>

3. Configurar env vars en Render (si no están)

- **STRIPE_CREDITOS_PREMIUM=100**
- **STRIPE_CREDITOS_PRO=500**
- Sin estas, `creditos_de_plan()` retorna None en producción y los webhooks no acreditan (lo cual es preferible a acreditar mal).

4. Continuar con T1.3.D (siguiente paso)

T1.3.D = endpoint `/internal/stripe-event` con verificación HMAC. Es el que recibe los eventos del bridge y llama a `procesar_evento_suscripcion`. No se implementó en T1.3.C (scope explícito del owner).